

Phanix: A 64-bit Memory-Safe Operating System and Interactive Shell

Project Proposal

Harshiv Bharat Patel

3120107

Griffith College, Cork
Ireland

Contents

1	Introduction	2
1.1	Project Overview	2
1.2	Motivation	2
1.3	Architectural Philosophy	2
1.4	Technical Specifications	2
2	Aims and Objectives	2
2.1	Primary Aim	2
2.2	Technical Objectives	2
2.3	Original Contribution	2
2.4	System Walkthrough: The ‘syscheck’ Example	3
3	Project Plan and Risk Analysis	3
3.1	Project Timeline (Weekly Milestones)	3
3.2	Risk Analysis and Mitigation	3
4	Evaluation Criteria	3
5	System Diagrams	4
6	References	4

1 Introduction

1.1 Project Overview

The project, titled **Phanix**, involves the design and implementation of a 64-bit operating system kernel from the ground up. Utilizing the Rust programming language, Phanix is architected as a monolithic kernel that prioritizes memory safety and system stability. The name is derived from Phanes, the Greek deity of creation, reflecting the project's goal of creating a functional software environment from the "primeval" state of bare-metal hardware. The final deliverable will be a bootable environment featuring a custom memory allocator, an interrupt-driven hardware layer, and an interactive command-line shell.

1.2 Motivation

Modern software development is increasingly decoupled from hardware. The motivation for Phanix is to bridge this gap by exploring fundamental laws of computing such as memory mapping and hardware communication without an existing OS. By using Rust, Phanix demonstrates how modern language paradigms prevent low-level vulnerabilities like buffer overflows.

1.3 Architectural Philosophy

Phanix follows a monolithic architecture where essential services such as memory management and hardware drivers reside within the kernel space. While originally envisioned as a microkernel, the current design prioritizes a functional bare-metal runtime that runs in a single privilege domain (CPU Ring 0). This approach ensures direct hardware access and simplified communication between system components.

1.4 Technical Specifications

- **Target Architecture:** x86_64 (64-bit Long Mode).
- **Memory Model:** Multi-level Paging with a Recursive Page Table mapping.
- **Concurrency:** Cooperative multitasking via an Async Executor.
- **Toolchain:** Rust `no_std` nightlies, LLVM, and the `bootimage` crate.
- **Legacy Hardware:** The project utilizes VGA text mode and the PS/2 keyboard interface for native QEMU compatibility and reduced driver complexity.

2 Aims and Objectives

2.1 Primary Aim

The primary goal is to develop a functional 64-bit bootable kernel that can manage hardware resources and execute tasks independently of any pre-existing operating system.

2.2 Technical Objectives

- **Hardware Abstraction:** Implement a VGA driver for visual output and a PS/2 keyboard driver for user input.
- **Interrupt Handling:** Configure an Interrupt Descriptor Table (IDT) to handle CPU exceptions.
- **Memory Architecture:** Implement a custom heap allocator to manage dynamic RAM allocation.
- **The Task System:** Build a cooperative task executor for background processes. This relies on tasks voluntarily yielding control, which is a known design trade-off for this scope.

2.3 Original Contribution

Beyond the standard implementation of a 64-bit kernel, Phanix will introduce an original contribution by implementing and benchmarking a dual-allocator strategy. The project will feature both a Bump Allocator and a Fixed-Size Block Allocator. I will perform an empirical comparison of these two strategies within the `syscheck` utility, measuring allocation speed and fragmentation overhead under simulated kernel workloads. This provides an analytical component not found in standard tutorials.

2.4 System Walkthrough: The ‘syscheck’ Example

1. **Input:** The user types `syscheck` into the shell.
2. **Allocation:** The kernel requests memory to store a temporary report string.
3. **Processing:** An Async Task queries the Page Table state and benchmarks the current active heap allocator.
4. **Output:** The task displays system health and performance metrics to the VGA Buffer.

3 Project Plan and Risk Analysis

3.1 Project Timeline (Weekly Milestones)

- **Week 1 to 2:** Freestanding Rust binary, VGA drivers, and basic CPU exception handling.
- **Week 3:** Contingency period for debugging IDT and Triple Fault issues.
- **Week 4 to 5:** Implementing Paging and the dual Heap Allocator strategy.
- **Week 6:** Integration of the Async Executor and Command Shell.
- **Week 7:** Comparative benchmarking and final evaluation.
- **Week 8:** Final report documentation and buffer for unforeseen bugs.

3.2 Risk Analysis and Mitigation

Risk	Impact	Mitigation Strategy
Timeline Slippage	High	Implementing weekly milestones and two dedicated contingency weeks.
Triple Fault Loops	High	Use QEMU with the <code>-d int</code> flag for detailed interrupt logging.
Complexity Overload	Medium	Maintain the monolithic design and prioritize the shell as the MVP.
Rust Toolchain Changes	Low	Pin the project to a specific nightly version in <code>rust-toolchain.toml</code> .
Memory Safety Leaks	Medium	Use Rust’s ownership model and avoid <code>unsafe</code> code outside of hardware abstraction.

Table 1: Phanix Project Risk Assessment

4 Evaluation Criteria

1. Boot successfully into a 64-bit environment.
2. Handle a Division-by-Zero exception without crashing.
3. Successfully allocate 1,000 objects using both allocator types.
4. Document performance differences between the two implemented allocators.
5. Respond to user commands via the shell in the multitasking executor.

5 System Diagrams

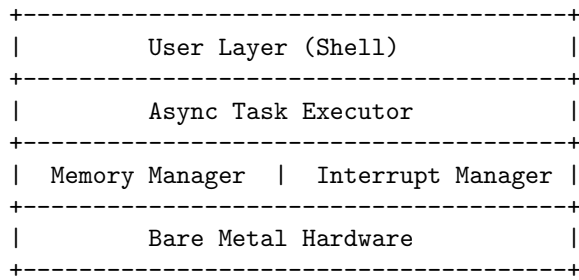


Figure 1: Architectural Layers of the Phanix Monolithic Kernel.

6 References

- Intel Corporation. (2026). *Intel 64 and ia-32 architectures software developer manuals* [Technical specifications for x86_64 interrupts, paging, and system instructions.]. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- Oppermann, P. (2024). *Writing an os in rust (edition 2)* [A comprehensive guide on implementing a 64-bit kernel using Rust's `no_std` environment.]. Retrieved February 19, 2026, from <https://os.phil-opp.com/>
- Tanenbaum, A. S., & Bos, H. (2014). *Modern operating systems* (4th) [Fundamental text on microkernel architecture and memory management principles.]. Pearson Education.
- The Rust Project Developers. (2026). *The rust reference: The `no_std` attribute*. Retrieved February 19, 2026, from https://doc.rust-lang.org/reference/attributes/codegen.html#the-no_std-attribute